

Kopya Kâğıdı + Çözümlü Sorular

Hashing · B-Tree · Graflar · En Kısa Yol · MST · İndirgeme

İlk bölüm tüm formüllerin **tek bakışta kopya kâğıdıdır**. İkinci bölümde her konudan **çözümlü sorular** var; her sorunun hemen altında adım adım çözümü ve "nasıl yaptık" açıklaması bulunur. Önce soruyu kapatıp kendin çözmeyi dene, sonra çözüme bak.

Bölüm A — Formül Kopya Kâğıdı

1) Hashing (Karma Tabloları)

Kavram	Formül / Kural
Hash fonksiyonu (bölme)	$h(k) = k \bmod m$
Yük faktörü	$\alpha = N / m$ (N: eleman, m: kutu)
Doğrusal yoklama	$p(k, i) = (h(k) + i) \bmod m$
Karesel yoklama	$p(k, i) = (h(k) + (i^2+i)/2) \bmod m$
Çift hashing	$p(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$
Zincirleme: beklenen zincir uzunluğu	$= \alpha \rightarrow \text{arama } 1+\alpha$
Açık adresleme: beklenen yoklama (başarısız)	$= 1 / (1 - \alpha)$
Açık adresleme: beklenen yoklama (başarılı)	$\approx (1/\alpha) \cdot \ln(1/(1-\alpha))$
Doğrusal yoklama (başarısız / başarılı)	$\frac{1}{2}(1+1/(1-\alpha)^2) \quad / \quad \frac{1}{2}(1+1/(1-\alpha))$
İşlem maliyeti	ortalama $O(1)$ · en kötü $O(N)$

2) B-Tree (minimum derece $t \geq 2$)

Kavram	Formül / Kural
Anahtar sayısı (kök hariç)	en az $t-1$ · en çok $2t-1$
Çocuk sayısı (kök hariç)	en az t · en çok $2t$
Dolu düğüm	$2t-1$ anahtar $\rightarrow$ eklemede bölünür
Bölmede yükselen anahtar	ortadaki (t.) anahtar $\rightarrow$ üst düğüme
Çocuk = anahtar + 1	k anahtar $\rightarrow$ k+1 çocuk
Yükseklik	$h \leq \log_t((n+1)/2) = O(\log_t n)$
Arama / ekleme / silme	$O(\log n)$

3) Graflar

Kavram	Formül / Kural
Tanım	$G = (V, E)$
El sıkışma teoremi	$\sum \deg(v) = 2 \cdot E $ (tek dereceli köşe sayısı çift)
Tam graf K_n kenar sayısı	$n(n-1)/2$
Kenar sayısı sınırı	$ E \leq V ^2$ (basit grafta $\leq V(V-1)/2$)
Euler turu (kapalı) koşulu	bağlı + tüm köşeler çift dereceli
Euler yolu (açık) koşulu	tam 2 köşe tek dereceli
Komşuluk matrisi / listesi (bellek)	$O(V^2)$ / $O(V+E)$

4) Gezme & En Kısa Yol

Algoritma	Kural / Karmaşıklık
Gevşetme (relax)	$\text{if dist}[u] + w(u, v) < \text{dist}[v]: \text{dist}[v] = \text{dist}[u] + w$
BFS / DFS	$O(V + E)$ · BFS=kuyruk, DFS=yığın · geri kenar=döngü
Dijkstra (negatif YOK)	$O((V + E) \cdot \log V)$ · greedy + min-heap
Bellman-Ford (negatif OK)	$O(V \cdot E)$ · V-1 tur · ekstra tur gevşerse negatif döngü
Floyd-Warshall (tüm çiftler)	$O(V^3)$ · $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$

5) Minimum Yayılan Ağaç (MST)

Kavram	Kural / Karmaşıklık
MST özellikleri	tüm köşeler · tam V-1 kenar · döngüsüz
Kesme özelliği (cut property)	bir kesmeyi geçen en hafif kenar MST'dedir
Kruskal	kenarları sırala + union-find · $O(E \log V)$
Kruskal türetmesi	$E \cdot \log E + E \cdot \alpha(V) = \theta(E \log E) = \theta(E \log V)$
Prim / Borůvka	$O(E \log V)$ · ağaç büyüt / bileşen birleştir

6) Paradigmalar, Sınırlar & İndirgeme

Kavram	Kural / Karmaşıklık
Böl ve Yönet	parçala-çöz-birleştir · Master teoremi ile analiz
Dinamik Programlama	örtüşen alt problem + optimal alt yapı (memoization)
Greedy	her adımda yerel en iyiyi seç
Arama / Sıralama alt sınırı	$O(\log n)$ / $O(n \log n)$
İndirgeme	$A \preceq_p B \cdot BEO(f) \rightarrow AE O(f) \cdot AE \Omega(f) \rightarrow BE \Omega(f)$
Karatsuba (çarpma)	$T(n) = 3T(n/2) + O(n) = O(n^{1.585})$
Strassen (matris çarpımı)	$O(n^{2.81})$

Master Teoremi (hızlı): $T(n) = a \cdot T(n/b) + O(n^d)$ için karşılaştır: a ile b^d . Eğer $a > b^d \rightarrow O(n^{\log_b a})$; $a = b^d \rightarrow O(n^d \log n)$; $a < b^d \rightarrow O(n^d)$.

7) En İyi / Ortalama / En Kötü Durum (toplu tablo)

Yöntem	En İyi	Ortalama	En Kötü	Bellek
Arama				
Doğrusal arama	$O(1)$	$O(n)$	$O(n)$	$O(1)$
İkili arama (sıralı veri)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Sıralama				
Kabarcık / Eklemeli sıralama	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Birleştirme (Merge) sıralama	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Hızlı (Quick) sıralama	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Yığın (Heap) sıralama	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Veri Yapıları (arama/ekleme/silme)				
Hash tablosu	$O(1)$	$O(1)$	$O(n)$	$O(n + m)$
B-Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
İkili Arama Ağacı (dengesiz)	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$
Graf Algoritmaları				
BFS / DFS	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$
Dijkstra (min-heap)	$O((V + E) \log V)$	$O((V + E) \log V)$	$O((V + E) \log V)$	$O(V)$
Bellman-Ford	$O(E) *$	$O(V \cdot E)$	$O(V \cdot E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^3)$	$O(V^3)$	$O(V^2)$
Kruskal / Prim (MST)	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$	$O(V + E)$

Okuma notları: **(1)** Quicksort en kötü $O(n^2)$ 'ye düşer (pivot hep en küçük/büyük seçilirse) — ama ortalama $O(n \log n)$ ve sabiti küçük olduğu için pratikte çok hızlıdır. **(2)** Hash ve dengesiz BST en kötü $O(n)$ olur (her şey tek kutuya / ağaç zincire dönerse); **B-Tree bunu engeller**, her durumda $O(\log n)$. **(3)** Graf algoritmalarında genelde üç durum aynıdır, çünkü algoritma yapısı girdiye göre dallanmaz; tüm köşe/kenarları işler. **(4) *** Bellman-Ford'un "en iyi"si, ilk turda hiç değişiklik olmazsa erken durma optimizasyonuyla mümkündür; standart hâli her zaman $O(V \cdot E)$ sayılır.

Bölüm B — Çözümlü Sorular

Her sorunun altında çözümü ve **yöntemin mantığı** var. Kendin denemek istersen çözümü elinle kapat.

HASHİNG

S1. $m = 11$ boyutlu bir hash tablosuna $h(k) = k \bmod 11$ ve **doğrusal yoklama** ile şu anahtarları sırayla ekle: **22, 1, 13, 11, 24, 33, 18, 42, 31**. Son tabloyu yaz.

ÇÖZÜM

Yöntem: Her anahtar için önce $k \bmod 11$ hesapla. O kutu doluyrsa, doğrusal yoklamayla bir sonraki kutuya (+1, +2, ... , sonu gelince başa dön) bak; ilk boş kutuya yerleştir.

- $22 \bmod 11 = 0 \rightarrow$ kutu **0** boş $\rightarrow$ yerleşti.
- $1 \bmod 11 = 1 \rightarrow$ kutu **1**.
- $13 \bmod 11 = 2 \rightarrow$ kutu **2**.
- $11 \bmod 11 = 0 \rightarrow$ 0 dolu $\rightarrow$ 1 dolu $\rightarrow$ 2 dolu $\rightarrow$ **3** boş $\rightarrow$ yerleşti.
- $24 \bmod 11 = 2 \rightarrow$ 2 dolu $\rightarrow$ 3 dolu $\rightarrow$ **4** boş.
- $33 \bmod 11 = 0 \rightarrow$ 0,1,2,3,4 dolu $\rightarrow$ **5** boş.
- $18 \bmod 11 = 7 \rightarrow$ kutu **7** boş.
- $42 \bmod 11 = 9 \rightarrow$ kutu **9** boş.
- $31 \bmod 11 = 9 \rightarrow$ 9 dolu $\rightarrow$ **10** boş.

0	1	2	3	4	5	6	7	8	9	10
22	1	13	11	24	33	–	18	–	42	31

Cevap: Yukarıdaki tablo. Dikkat: 11, 33 gibi anahtarlar 0'a düşüp uzun bir "dolu blok" oluşturduğu için sonraki eklemeler de itildi — bu **birincil kümelenmedir**.

HASHİNG

S2. Bir açık adresleme tablosunda $m = 1000$ kutu var ve $N = 750$ eleman ekli. Başarısız ve başarılı bir aramanın **beklenen yoklama sayısı** yaklaşık kaçtır?

ÇÖZÜM

Yöntem: Önce yük faktörü $\alpha = N/m$ 'i bul, sonra formüllere koy.

- $\alpha = 750 / 1000 = 0.75$
- Başarısız arama: $1/(1-\alpha) = 1/(1-0.75) = 1/0.25 = 4$ yoklama.
- Başarılı arama: $(1/\alpha) \cdot \ln(1/(1-\alpha)) = (1/0.75) \cdot \ln(4) = 1.333 \times 1.386 \approx 1.85$ yoklama.

Cevap: Başarısız ≈ 4 , başarılı ≈ 1.85 yoklama. **Neden mantıklı?** Tablo %75 dolu; boş kutu bulmak (başarısız arama) doluyken daha zor, o yüzden 4; var olanı bulmak ise ortalama daha kısa, ~ 1.85 .

B-TREE

S3. Minimum derecesi $t = 4$ olan bir B-Tree'de bir düğüm (kök hariç) en az/en çok kaç **anahtar** ve kaç **çocuk** taşır? Düğüm hangi anahtar sayısında "dolu" sayılır?

ÇÖZÜM

Yöntem: Doğrudan kapasite kuralları: anahtar $\in [t-1, 2t-1]$, çocuk $\in [t, 2t]$, dolu $= 2t-1$.

- Anahtar: en az $t-1 = 3$, en çok $2t-1 = 7$.
- Çocuk: en az $t = 4$, en çok $2t = 8$.
- Dolu: $2t-1 = 7$ anahtar olunca. 8.'yi eklemeye çalışınca bölünür.

Cevap: 3–7 anahtar, 4–8 çocuk, dolu = 7 anahtar. (Çocuk sayısı hep anahtar sayısının 1 fazlasıdır.)

B-TREE

S4. $t = 3$ olan bir B-Tree'de bir yaprak **dolu**: $[10, 20, 30, 40, 50]$. Bu yaprağa **25** eklenmek isteniyor. Bölme nasıl olur? Hangi anahtar yukarı çıkar?

ÇÖZÜM

Yöntem: $t=3$ için dolu $= 2t-1 = 5$ anahtar. Dolu düğüme ekleme gelince **bölünür**: ortadaki ($t = 3$.) anahtar yukarı çıkar, kalan ikiye anahtar sol ve sağ düğüme paylaşılır; yeni anahtar doğru tarafa yerleşir.

- Dolu düğüm $[10, 20, 30, 40, 50] \rightarrow$ ortadaki **30** (3. anahtar) yukarı, köke çıkar.
- Sol düğüm = ilk 2 anahtar: $[10, 20]$ Sağ düğüm = son 2 anahtar: $[40, 50]$.
- Eklenecek **25**, 30'dan küçük $\rightarrow$ sol düğüme sıralı girer $\rightarrow [10, 20, 25]$.

Cevap: **30** köke yükselir; sol yaprak $[10, 20, 25]$, sağ yaprak $[40, 50]$ olur. Ağaç bir seviye büyür ama dengeli kalır.

B-TREE

S5. $t = 3$ ve $n = 100$ anahtar içeren bir B-Tree'nin yüksekliği en fazla kaçtır?

ÇÖZÜM

Yöntem: Yükseklik formülünü kullan: $h \leq \log_t((n+1)/2)$.

- $(n+1)/2 = 101/2 = 50.5$
- $h \leq \log_3(50.5) = \ln(50.5)/\ln(3) = 3.92/1.099 \approx 3.57$
- h tam sayı olmalı $\rightarrow h \leq 3$.

Cevap: En fazla **3**. **Yorum:** 100 anahtarı saklamak için 3 seviye yeter — ikili ağaçta ($\log_2 100 \approx 7$) iki katından fazla derin olurdu. t büyüdükçe ağaç sığlaşır.

GRAF

S6. Bir grafın köşe dereceleri şöyle: $A=3$, $B=2$, $C=3$, $D=2$, $E=4$. Bu grafta kaç **kenar** vardır? Ayrıca tam graf K_6 'da kaç kenar olur?

ÇÖZÜM

Yöntem: El sıkışma teoremi $\sum \deg = 2|E|$ ve tam graf formülü $n(n-1)/2$.

- $\sum \deg = 3+2+3+2+4 = 14$
- $2|E| = 14 \rightarrow |E| = 7$
- $K_6: 6 \cdot 5/2 = 15$ kenar.

Cevap: 7 kenar; $K_6 = 15$ kenar. (Derece toplamı her zaman çifttir, çünkü her kenar 2 dereceye katkı yapar.)

GRAF

S7. Dört kara parçası 7 köprüyle bağlı; köşe dereceleri $3, 3, 3, 5$ (Königsberg). Her köprüden tam bir kez geçip başlangıca dönülebilir mi (Euler turu)? Hiç olmazsa açık bir Euler **yolu** var mı?

ÇÖZÜM

Yöntem: Euler turu için **tüm** köşeler çift dereceli olmalı. Euler yolu için **en çok 2** köşe tek dereceli olabilir.

- Dereceler: $3, 3, 3, 5 \rightarrow$ **hepsi tek**. Tek dereceli köşe sayısı $= 4$.
- Euler turu: 0 tek dereceli köşe gerekir $\rightarrow 4$ var $\rightarrow$ **imkânsız**.
- Euler yolu: en çok 2 tek dereceli köşe gerekir $\rightarrow 4$ var $\rightarrow$ **o da imkânsız**.

Cevap: Ne Euler turu ne de Euler yolu vardır. Euler 1736'da tam bunu kanıtladı; graf teorisi böyle doğdu.

GEZME / DFS

S8. Yönlü graf: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$, $C \rightarrow D$. A 'dan DFS başlat. Hangi kenar **geri kenardır** ve bu ne anlama gelir?

ÇÖZÜM

Yöntem: DFS'i ilerlet; ilk kez ulaşılan kenarlar **ağaç kenarıdır**. Daha önce keşfedilip hâlâ "açık" (atası olan) bir köşeye giden kenar **geri kenardır** $\rightarrow$ döngü işareti.

- $A \rightarrow B$: B yeni $\rightarrow$ ağaç kenarı. $B \rightarrow C$: C yeni $\rightarrow$ ağaç kenarı.
- $C \rightarrow A$: A zaten DFS ağacında **C'nin atası** ($A \rightarrow B \rightarrow C$ yolundan) $\rightarrow$ **geri kenar**.
- $C \rightarrow D$: D yeni $\rightarrow$ ağaç kenarı.

Cevap: $C \rightarrow A$ geri kenardır ve $A \rightarrow B \rightarrow C \rightarrow A$ **döngüsünü** ortaya çıkarır. Kural: **geri kenar bulduysan grafta döngü vardır**.

EN KISA YOL — DIJKSTRA

S9. Yönsüz ağırlıklı graf, kenarlar: $A-B:2$, $A-C:4$, $B-C:1$, $B-D:7$, $C-D:3$, $C-E:5$, $D-E:2$. Kaynak **A**. Tüm köşelere en kısa mesafeleri Dijkstra ile bul.

ÇÖZÜM

Yöntem: $dist[A]=0$, gerisi ∞ . Her adımda **kesinleşmemiş en küçük dist'li köşeyi** seç, kesinleştir, komşularını gevşet. Bir köşe kesinleşince mesafesi değişmez.

Kesinleşen	A	B	C	D	E
başlangıç	0	∞	∞	∞	∞
A (0)	0	2	4	∞	∞
B (2)	0	2	3 (2+1)	9 (2+7)	∞
C (3)	0	2	3	6 (3+3)	8 (3+5)
D (6)	0	2	3	6	8 (6+2=8, eşit)
E (8)	0	2	3	6	8

Adımlar: A'dan B(2),C(4) açılır. En küçük B(2) kesinleşir; B üzerinden $C: 2+1=3$ 'e düşer, $D=9$ olur. C(3) kesinleşir; D $3+3=6$ 'ya düşer, $E=8$ olur. D(6) kesinleşir; E için $6+2=8$, mevcut 8'e eşit, değişmez. E(8) kesinleşir.

Cevap: $A \rightarrow A:0$, $A \rightarrow B:2$, $A \rightarrow C:3$, $A \rightarrow D:6$, $A \rightarrow E:8$. (Not: $A \rightarrow C$ doğrudan 4 yerine B üzerinden 3 bulundu; greedy seçim sayesinde her köşe bir kez kesinleşti.)

EN KISA YOL — BELLMAN-FORD

S10. Yönlü graf, kenarlar (her turda bu sırayla): $A \rightarrow B:4$, $A \rightarrow C:5$, $B \rightarrow C:-3$, $C \rightarrow D:2$, $B \rightarrow D:6$. Kaynak A. Mesafeleri bul ve negatif döngü olup olmadığını söyle.

ÇÖZÜM

Yöntem: $dist[A]=0$, gerisi ∞ . $V-1=3$ tur boyunca **tüm kenarları** sırayla gevşet. Sonra bir tur daha dene; hâlâ gevşeme olursa negatif döngü vardır.

Tur sonu	A	B	C	D	Bu turda
başlangıç	0	∞	∞	∞	—
Tur 1	0	4	1	3	$A \rightarrow B=4$, $A \rightarrow C=5$, $B \rightarrow C=4-3=1$, $C \rightarrow D=1+2=3$
Tur 2	0	4	1	3	hiçbir kenar gevşemedi $\rightarrow$ bitti

Tur 1'de $A \rightarrow C$ önce 5 yaptı ama hemen ardından $B \rightarrow C$ kenarı $4-3=1$ ile onu düşürdü (negatif kenarın etkisi). $C \rightarrow D$ de $1+2=3$ oldu. Tur 2'de değişiklik yok.

Cevap: A:0, B:4, C:1, D:3. Ekstra turda hiçbir mesafe değişmediği için **negatif döngü yoktur**. (Dijkstra bu grafta $B \rightarrow C:-3$ yüzünden yanlış yapardı; Bellman-Ford doğru buldu.)

EN KISA YOL — FLOYD-WARSHALL

S11. Yönlü graf (3 köşe), kenarlar: $1 \rightarrow 2:4$, $2 \rightarrow 3:1$, $1 \rightarrow 3:11$, $3 \rightarrow 1:2$. Tüm çiftler arası en kısa mesafe matrisini Floyd-Warshall ile bul.

ÇÖZÜM

Yöntem: Başlangıç matrisini doldur (köşegen 0, kenar yoksa ∞). Sonra her ara durak $k=1,2,3$ için tüm (i,j) 'yi $d[i][j]=\min(d[i][j], d[i][k]+d[k][j])$ ile güncelle.

Başlangıç:

	1	2	3
1	0	4	11
2	∞	0	1
3	2	∞	0

k=1 (1 üzerinden): $d[3][2]=d[3][1]+d[1][2]=2+4=6 \rightarrow$ güncellenir.

k=2 (2 üzerinden): $d[1][3]=d[1][2]+d[2][3]=4+1=5 < 11 \rightarrow$ 5 olur.

k=3 (3 üzerinden): $d[2][1]=d[2][3]+d[3][1]=1+2=3 \rightarrow$ güncellenir.

Sonuç:

	1	2	3
1	0	4	5
2	3	0	1
3	2	6	0

Cevap: Yukarıdaki son matris. Örn. $2 \rightarrow 1$ doğrudan yok ama $2 \rightarrow 3 \rightarrow 1 = 1+2 = 3$ ile en kısası bulundu; $1 \rightarrow 3$ doğrudan 11 yerine $1 \rightarrow 2 \rightarrow 3 = 5$ oldu.

MST — KRUSKAL

S12. 5 köşeli (A,B,C,D,E) graf. Kenarlar: $B \rightarrow C:1$, $C \rightarrow E:1$, $A \rightarrow B:2$, $B \rightarrow E:2$, $A \rightarrow C:3$, $B \rightarrow D:3$, $D \rightarrow E:5$. Kruskal ile MST'yi ve toplam maliyeti bul.

ÇÖZÜM

Yöntem: Kenarları maliyete göre sırala. En ucuzdan başla; bir kenar **döngü oluşturmuyorsa** (iki ucu farklı kümedeyse) ekle, oluşturuyorsa atla. $V-1=4$ kenar olunca dur.

Kenar	Maliyet	Karar
B-C	1	EKLE $\rightarrow \{B,C\}$
C-E	1	EKLE $\rightarrow \{B,C,E\}$
A-B	2	EKLE $\rightarrow \{A,B,C,E\}$
B-E	2	B,E aynı kümede $\rightarrow$ döngü $\rightarrow$ ATLA
A-C	3	A,C aynı kümede $\rightarrow$ döngü $\rightarrow$ ATLA
B-D	3	EKLE $\rightarrow \{A,B,C,D,E\} \rightarrow$ 4 kenar, DUR

Cevap: MST = $\{B-C, C-E, A-B, B-D\}$, toplam maliyet = $1+1+2+3 = 7$. Döngü kontrolü Union-Find ile yapılır (iki köşe aynı kökteyse aynı kümededir).

KARMAŞIKLIK

S13. Kruskal'ın karmaşıklığı neden $O(E \log E)$ ve $O(E \log V)$ olarak **aynı anda** yazılabilir? Göster.

DERİN

ÇÖZÜM

Yöntem: Maliyeti yaz, ardından kenar sayısı ile köşe sayısı arasındaki $E \leq V^2$ bağıntısını kullan.

- Kruskal = sıralama ($E \cdot \log E$) + Union-Find ($E \cdot \alpha(V)$). Ters Ackermann $\alpha(V) \approx$ sabit (4'ten küçük) $\rightarrow$ toplam $\Theta(E \cdot \log E)$.
- Bir grafta $E \leq V^2$. O hâlde $\log E \leq \log(V^2) = 2 \cdot \log V$.
- Sabit çarpan (2) Θ içinde yutulur: $\Theta(E \cdot \log E) = \Theta(E \cdot 2 \log V) = \Theta(E \cdot \log V)$.

Cevap: $E \leq V^2$ olduğu için $\log E$ ile $\log V$ yalnızca sabit bir çarpanla (2) farklıdır; Θ gösteriminde bu fark kaybolur. Bu yüzden ikisi de doğrudur: $\Theta(E \log E) = \Theta(E \log V)$.

İNDİRGEME / BÖL-YÖNET

S14. Karatsuba çarpma algoritmasının özyineleme bağıntısı $T(n) = 3 \cdot T(n/2) + O(n)$ 'dir. Karmaşıklığı bul ve neden naif $O(n^2)$ 'den hızlı olduğunu açıkla.

ÇÖZÜM

Yöntem: Master teoremiyle çöz. $T(n) = a \cdot T(n/b) + O(n^d)$: burada $a=3$, $b=2$, $d=1$.

- Karşılaştır: $a = 3$ ile $b^d = 2^1 = 2$. $a > b^d$ ($3 > 2$).
- Bu durumda sonuç: $O(n^{\log_b a}) = O(n^{\log_2 3}) \approx O(n^{1.585})$.
- Karşılaştırma: $n^{1.585} < n^2$ (büyük n için çok daha küçük).

Cevap: $O(n^{1.585})$. Karatsuba, sayıları ikiye bölüp **4 yerine 3 çarpma** yaptığı için üs 2'den 1.585'e iner; bu yüzden büyük sayılarda okul yöntemindeki $O(n^2)$ 'den belirgin hızlıdır. (Bu, "çarpma problemini daha küçük çarpma alt problemlerine indirgemenin" klasik örneğidir.)